

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Intermediate Level)

Assessment 5: Web Application Vulnerability Testing on DVWA

1. Introduction

The objective of this task is to set up a vulnerable web application environment and perform security testing on common vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS).

These vulnerabilities are part of the **OWASP Top 10**, which are widely exploited in real-world cyber attacks.

In this task, **Metasploitable 2** was used as the target system, which contains intentionally vulnerable services including **Damn Vulnerable Web Application (DVWA)**.

2. About Vulnerabilities

SQL Injection (SQLi)

SQL Injection allows attackers to manipulate database queries to gain unauthorized access to sensitive data.

Cross-Site Scripting (XSS)

XSS allows attackers to inject malicious scripts into web pages.

Types of XSS:

- ⑩ Reflected XSS (Non-persistent)
- ⑩ Stored XSS (Persistent)

3. Lab Environment

- ⑩ Attacker Machine: Kali Linux
- ⑩ Target Machine: Metasploitable 2
- ⑩ Target IP Address: 192.168.56.101
- ⑩ Web Application: DVWA
- ⑩ Web Server: Apache
- ⑩ Database: MySQL
- ⑩ Tools Used: Browser, Terminal, **SQLMap**
- ⑩ Network Type: Virtual Lab (Host-only Adapter)
- ⑩ Security Level: Low

4. Methodology

The testing process was performed in multiple stages including lab setup and vulnerability exploitation.

4.1 Lab Setup (Metasploitable + DVWA)

The virtual lab environment was created using VirtualBox with Kali Linux as the attacker machine and Metasploitable 2 as the target.

The target IP address was identified using:

```
ifconfig
```

DVWA was accessed through:

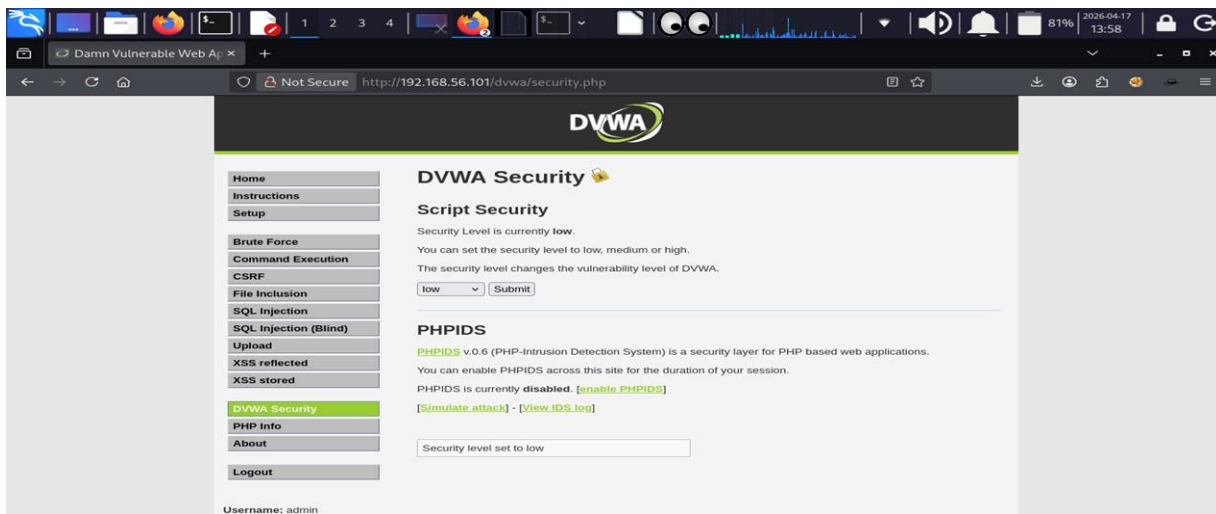
```
http://192.168.56.101/dvwa
```

Login credentials:

Username: admin

Password: password

Security level was set to **Low**.



4.2 SQL Injection

4.2.1 Identifying SQL Injection Point

1'

Observation:

SQL error confirms vulnerability.



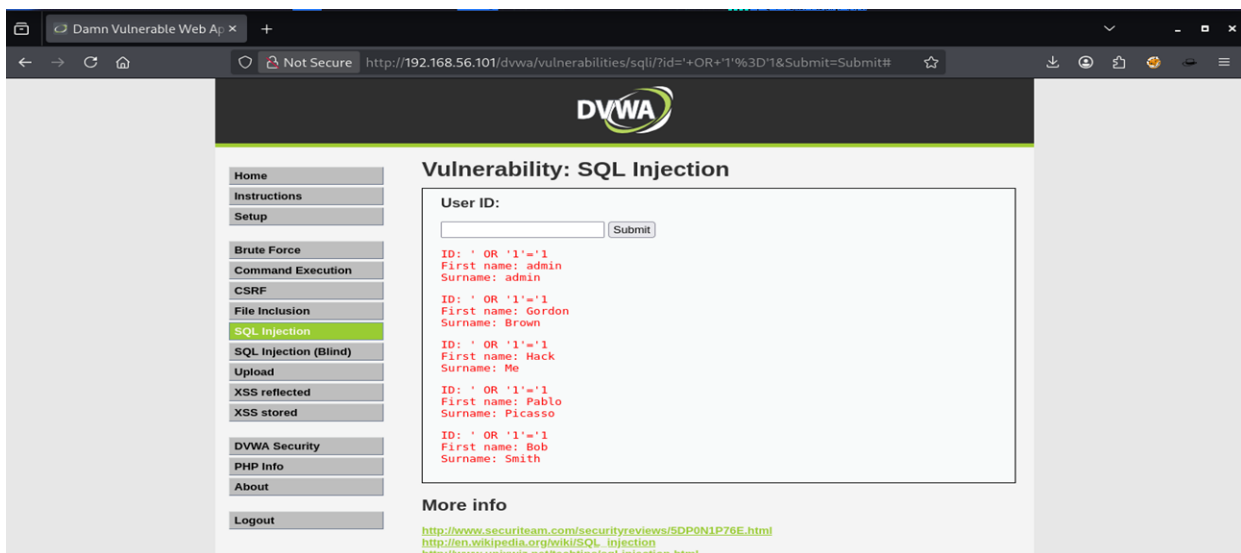
SQL syntax error generated after injecting invalid input (1'), confirming improper input validation.

4.2.2 Authentication Bypass

' OR '1'='1

Observation:

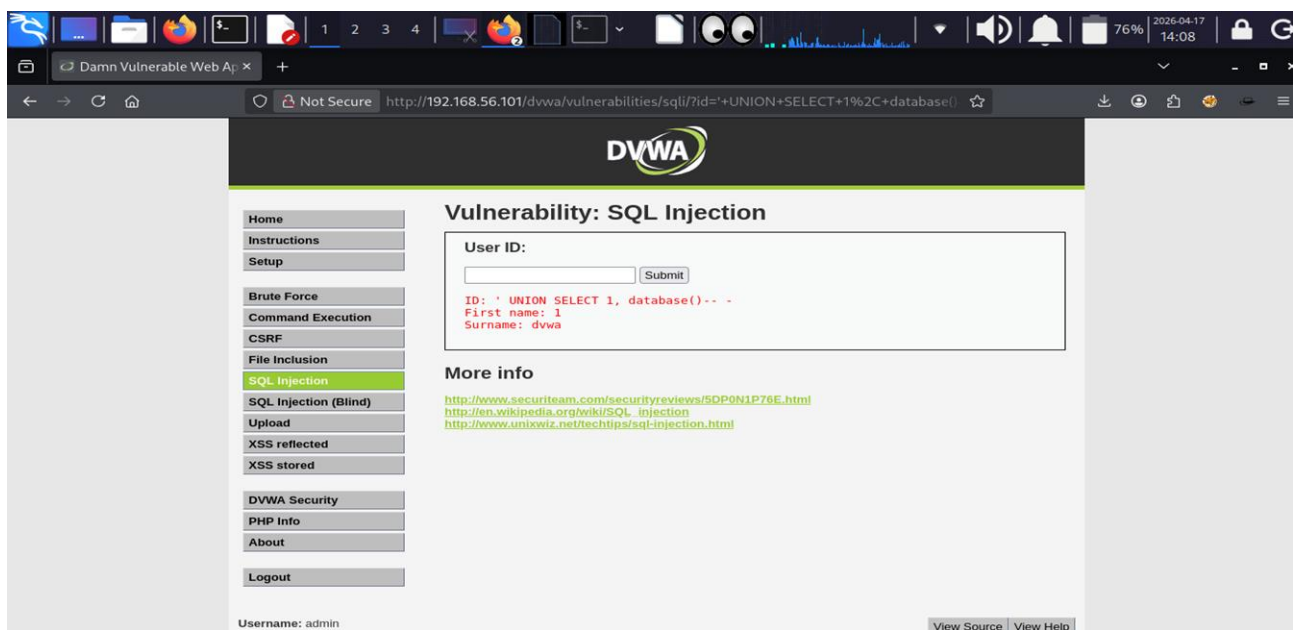
Login bypass successful.



Login bypass using SQL Injection payload that forces condition to always evaluate TRUE.

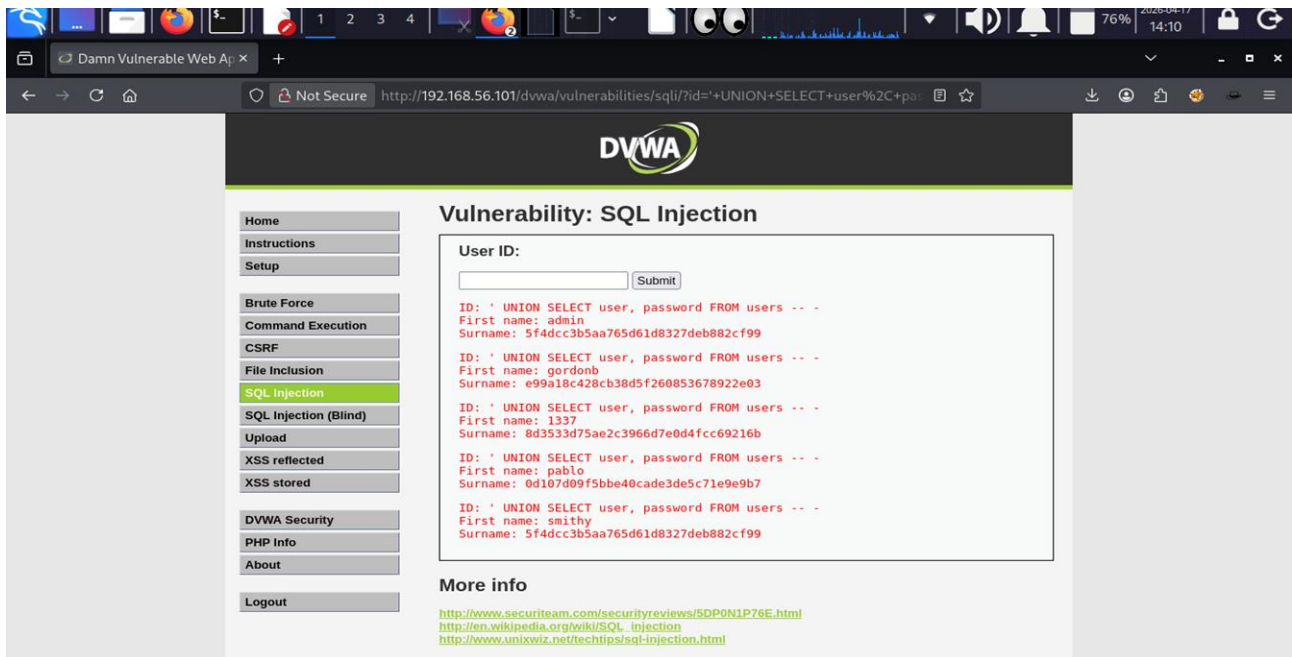
4.2.3 UNION-Based SQL Injection

' UNION SELECT 1, database() -- -



Database name extracted using UNION-based SQL Injection.

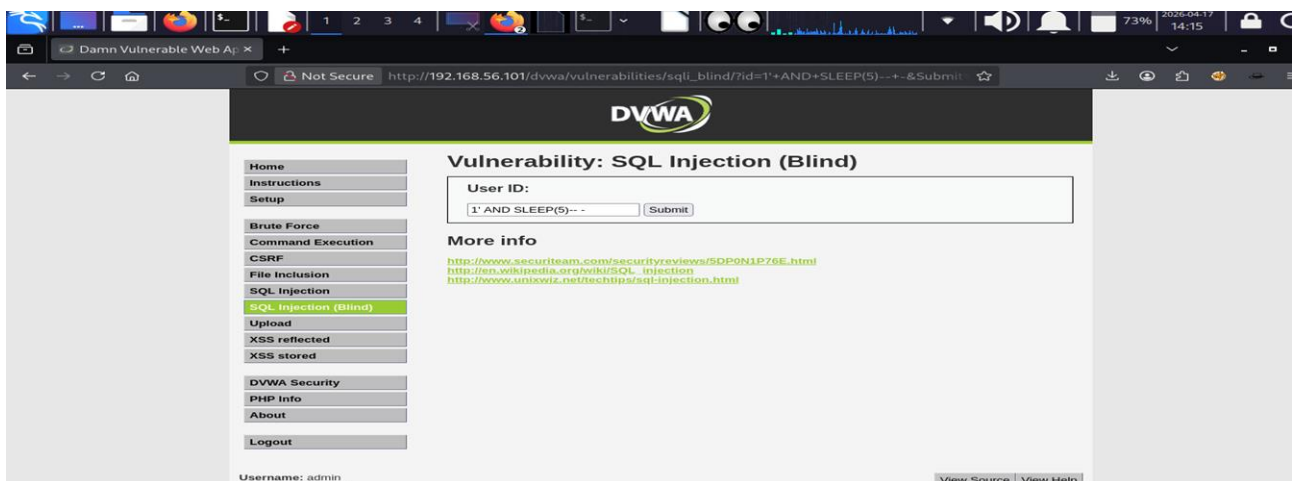
' UNION SELECT user, password FROM users-- -



User credentials retrieved from database, demonstrating data exposure risk.

4.2.4 Blind SQL Injection

1' AND SLEEP(5)-- --



Time delay introduced using SLEEP() confirms blind SQL Injection vulnerability.

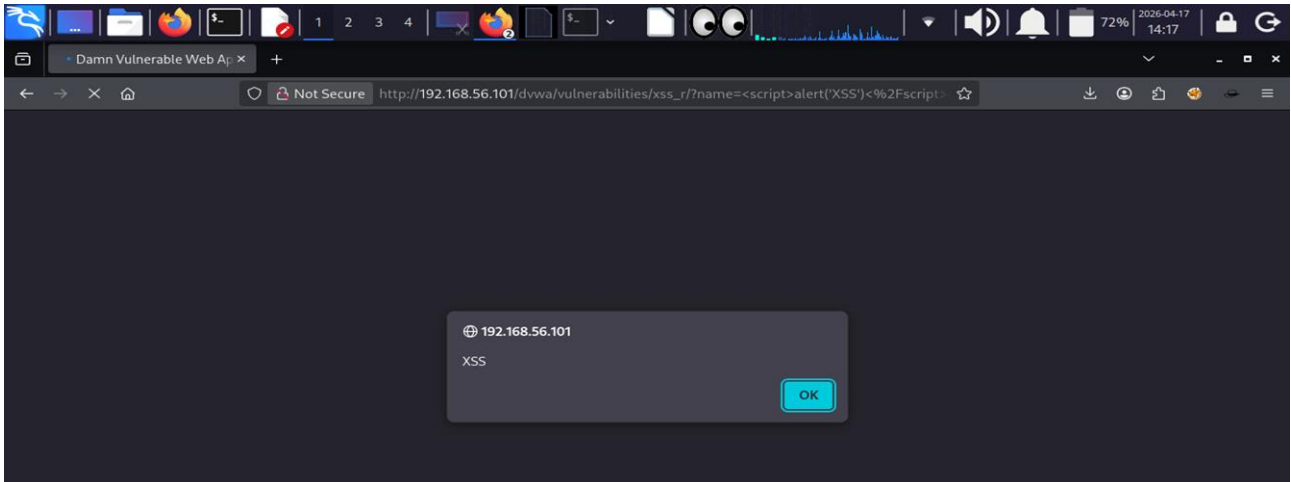
4.3 Reflected XSS

Payload Used

```
<script>alert(' XSS' )</script>
```

Observation:

Script executes immediately.



Reflected XSS payload executed instantly due to lack of input sanitization.

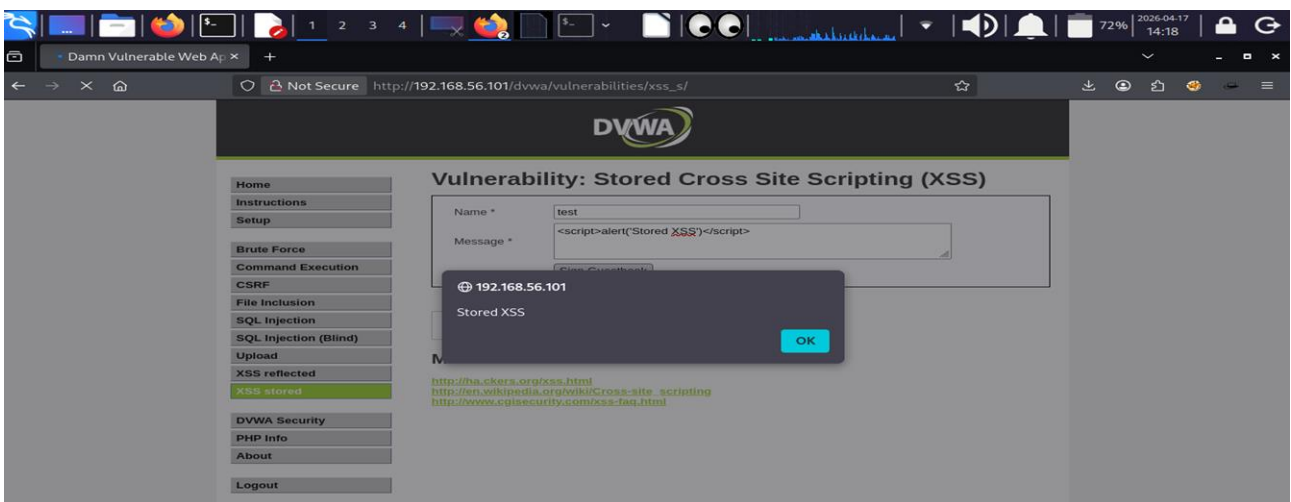
4.4 Stored XSS

Payload Used

```
<script>alert(' Stored XSS' )</script>
```

Observation:

Script executes every time page loads.



Stored XSS payload persists in database and executes on every page load.

5. Results and Analysis

5.1 Vulnerabilities Identified

Vulnerability	Type	Risk
SQL Injection	Server-side	Critical
Reflected XSS	Client-side	High
Stored XSS	Persistent	Critical

5.2 Data Exposure

- Ⓢ Database name
- Ⓢ Table structure
- Ⓢ User credentials

5.3 Root Cause Analysis

- Ⓢ No input validation
- Ⓢ Direct execution of user input
- Ⓢ No output encoding
- Ⓢ Weak security configuration

6. Vulnerability Assessment

The application is highly vulnerable and easily exploitable due to insecure coding practices.

7. Security Implications

- Ⓢ Unauthorized access
- Ⓢ Data breaches
- Ⓢ Session hijacking
- Ⓢ Credential theft

8. Security Recommendations

SQL Injection:

- Ⓢ Use prepared statements
- Ⓢ Parameterized queries

XSS:

- ⑩ Input validation
- ⑩ Output encoding
- ⑩ Content Security Policy (CSP)

General:

- ⑩ Use Web Application Firewall (WAF)
- ⑩ Restrict database access
- ⑩ Avoid displaying error messages

9. Skills Acquired

- ⑩ Setting up virtual lab environment
- ⑩ SQL Injection exploitation
- ⑩ Reflected and Stored XSS attacks
- ⑩ Using **SQLMap**
- ⑩ Web security testing

10. Conclusion

This task provided hands-on experience in exploiting SQL Injection and XSS vulnerabilities. The experiment demonstrated how insecure applications can be compromised due to improper input handling.

11. Final Outcome

- ⑩ Successfully exploited SQL Injection
- ⑩ Performed Reflected and Stored XSS
- ⑩ Extracted sensitive data
- ⑩ Learned mitigation techniques